J Sched (2017) 20:25–42
DOI 10.1007/s10951-016-0498-5

Scheduling shipments in closed-loop sortation conveyors

Dirk Briskorn[1] · **Simon Emde**[2] · **Nils Boysen**[3]

Published online: 18 October 2016

Abstract At the very core of most automated sorting systems— for example, at airports for baggage handling and in parcel distribution centers for sorting mail—we find closed-loop tilt tray sortation conveyors. In such a system, trays are loaded with cargo as they pass through loading stations, and are later tilted upon reaching the outbound container dedicated to a shipment's destination. This paper addresses the question of whether the simple decision rules typically applied in the real world when deciding which parcel should be loaded onto what tray are, indeed, a good choice. We formulate a short-term deterministic scheduling problem where a finite set of shipments must be loaded onto trays such that the makespan is minimized. We consider different levels of flexibility in how to arrange shipments on the feeding conveyors, and distinguish between unidirectional and bidirectional systems. In a comprehensive computational study, we compare these sophisticated optimization procedures with widespread rules of thumb, and find that the latter perform surprisingly well. For almost all problem settings, some priority rule can be identified which leads to a low-single-digit optimality gap. In addition, we systematically evaluate the performance gains promised by different sorter layouts.

Keywords Logistics · Transshipment · Sortation conveyor · Scheduling

✉ Nils Boysen
nils.boysen@uni-jena.de
http://www.om.uni-jena.de/

Dirk Briskorn
briskorn@wiwi.uni-wuppertal.de
http://www.prodlog.uni-wuppertal.de/

Simon Emde
emde@bwl.tu-darmstadt.de
http://www.or.wi.tu-darmstadt.de/

[1] Professur für BWL, insbesondere Produktion und Logistik, Bergische Universität Wuppertal, Rainer-Gruenter-Str. 21, 42119 Wuppertal, Germany

[2] Fachgebiet Management Science / Operations Research, Technische Universität Darmstadt, Hochschulstraße 1, 64289 Darmstadt, Germany

[3] Lehrstuhl für Operations Management, Friedrich-Schiller-Universität Jena, Carl-Zeiß-Straße 3, 07743 Jena, Germany

1 Introduction

Fully automated sortation processes play a crucial role in many distribution networks and supply chains. On the one hand, these systems promise much higher reliability than manual operations. For instance, the U.S. Department of Transportation (2009) reports a range of 1.42 to 9.17 when ranking 19 U.S. airlines with regard to mishandled baggage per 1000 passengers. Clearly, any lost or delayed baggage causes loss of goodwill, and thus reliable and fully automated luggage handling has become a critical success factor in the airline industry. Moreover, automation promises much higher efficiency of sortation processes. For instance, the airport hub in Leipzig (Germany), which is one of three main hubs worldwide in postal service provider DHL's express logistics network, sorts up to 60,000 parcels and 36,000 documents per h. Such huge throughput is enabled by a fully automated system of sortation conveyors (SCs), where the main SC has a length of 6500 m and serves up to 260 stations for processing of air freight containers (DHL 2008).

Fully automated sortation systems require huge investments, and are thus often bottleneck resources. At the airport hub in Leipzig, for instance, €70 million of a total investment of €300 million is allotted to the conveyor system (DHL

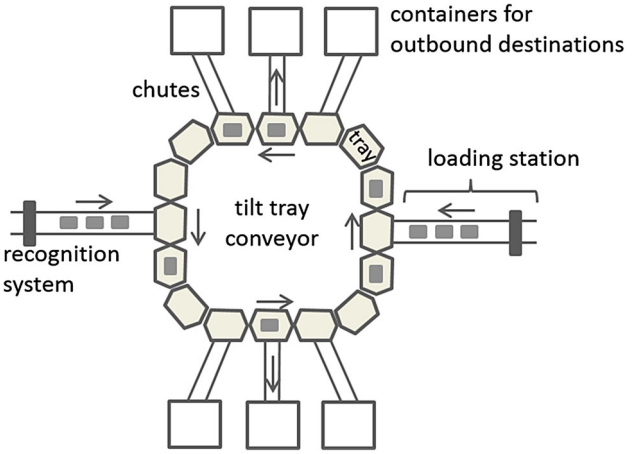

Fig. 1 Schematic layout of a closed-loop tilt tray sortation conveyor

2008). Therefore, careful planning of the layout and operational processes of the SC is critical.

1.1 System configuration and sortation processes

At the heart of most automated sorting systems we find closed-loop tilt tray SCs (see Fig. 1). An SC consists of a set of linked trays arranged in a closed loop. Without loss of generality, throughout this paper, we can picture this loop as a circle. This loop of trays is continuously moving either clockwise or counterclockwise. We will refer to it hereinafter as the main conveyor . In Fig. 1, each tray is depicted as a shaded hexagon. The SC in this case has 16 trays moving counterclockwise (as shown by arrows). Note that there are also bidirectional SCs, such as the one installed at the aforementioned airport hub in Leipzig, which consist of multiple loops arranged one on top of the other, moving in opposite directions.

An SC also features one or multiple loading stations and many outbound containers which feed shipments onto the main conveyor and collect them after the sortation process. The flow of shipments through these system elements is as follows: First, shipments are identified, for instance, by scanning a bar code attached to a piece of luggage (Abdelghany et al. 2006) or by identifying a handwritten address on a parcel by some OCR software (Fedtke and Boysen 2014). After identification, a switch system successively loads the shipments queuing at a loading station on free-tilt trays, so that a one-to-one mapping between shipments and trays is achieved and communicated to the background information system. In Fig. 1, we see two loading stations, on the left and the right of the main conveyor, respectively. Shipments travel through them (and the recognition system they are equipped with) toward the trays. They are typically loaded onto a tray simply by pushing them from the loading station onto a passing tray.

On their trays, shipments travel along the main conveyor until reaching their respective gravity chutes. The tray is then tilted, and the shipment slides to some container collecting all goods for a certain destination. Such a container may be a dead-end conveyor (pier) for collecting all baggage for a specific flight (Abdelghany et al. 2006) or an outbound truck parked at a dock door of a cross-docking terminal (Boysen and Fliedner 2010). In Fig. 1, we see three chutes above and three chutes below the main conveyor.

Of course, there exist other SCs whose system elements deviate slightly from the above description. Instead of tiltable trays, they can also be equipped with a crossbelt, which moves the loaded shipment perpendicular to the flow direction of the main conveyor once the dedicated chute is passed. In addition, gravity chutes can be replaced by a belt conveyor, and different kinds of recognition systems can be applied, such as bar code, camera recognition, or radio frequency identification. However, the general sortation process considered in this paper is applied across a wide range of facilities, including airports for baggage handling (Abdelghany et al. 2006), postal distribution centers (Fedtke and Boysen 2014), retail distribution centers (Johnson and Lofgren 1994; Johnson 1998), and cross-docking terminals (Boysen and Fliedner 2010).

1.2 Decision problems and literature review

There are two main decision problems to be solved regarding the operational sortation process, as follows:

- The existing literature on sortation processes deals largely with the *assignment of outbound destinations to containers* in order to reduce the travel distance of shipments inside a terminal. In Fig. 1, six outbound containers can be placed in parallel. Preferably, shipments entering the main conveyor from the right side should have a destination container which is placed on one of the three positions at the top of Fig. 1. They then have a relatively short travel time on the main conveyor, and do not block trays passing the opposite loading station.
  Considering the specific characteristics of the respective applications, optimization procedures for this assignment decision can be found for SCs at airports (Abdelghany et al. 2006; Asco et al. 2014), parcel distribution centers (Fedtke and Boysen 2014; Werners and Wülfing 2010) retail distribution centers (Johnson 1998; Meller 1997), and cross-docking terminals (Bozer and Carlo 2008; Gue 1999; Tsui and Chang 1990; Tsui and Chang 1992). In some applications, e.g., retail distribution centers and airports, the destination assignments change frequently throughout the day, once other orders (Johnson 1998; Meller 1997) or flights (Abdelghany et al. 2006;

Asco et al. 2014) are completed. In other facilities, e.g., postal distribution centers and cross-docking terminals, the assignment of outbound destinations to dock doors is typically fixed over a longer period of time, e.g., a month in Boysen and Fliedner (2010), since they face an ongoing stream of shipments to the destinations. Similarly, the scheduling and allocation of inbound resources can be used to improve a sortation facility's performance (see, e.g., Haneyah et al (2014)). In Clausen et al. (2015), both aspects—scheduling of inbound trailers and allocation of destinations to main sorters—are coordinated in a holistic manner. We assume, however, that the destination assignment problem has already been solved, and that all outbound destinations remain unaltered during the planning horizon. Since our problem is very short-term in nature, i.e., solved over a period of less than 1 h, this assumption does not preclude any aforementioned SC application.

- Once shipments queue in front of their loading stations, the very short-term *sorter scheduling problem* is to decide which parcel is to be loaded onto what passing tray. Clearly, for a single loading station and a single (unidirectional) SC, there is no choice but to load each shipment whenever a free tray approaches. With multiple stations, however, a shipment loaded now may block another when passing by a later station. Consider Fig. 1 as an example. If a shipment approaches the main conveyor through the right loading station, and has a destination container which is placed on one of the three positions at the bottom of Fig. 1, it then necessarily travels past the left loading station (recall that the main conveyor moves counterclockwise). Thus, the tray this shipment travels on is prevented from being loaded with a—potentially urgent—shipment in front of the left loading station at that time. With parallel main conveyors rotating contrariwise, the direction any parcel should take becomes an additional decision task.

  In the real world, the sorter scheduling problem is typically seen as a real-time control problem, and simple local decision rules are applied, such as the shortest distance rule (if parallel SCs are applied, Fedtke and Boysen (2014)). However, this is not necessarily the case. In major terminals (such as hub terminals of parcel carriers), all shipments are identified as they are unloaded from the vehicle or container they arrive in. They are then put on a conveyor which moves them toward the SC with constant velocity. On their way, shipments are constantly tracked such that their positions and assigned paths through the terminal are known, and quasi-deterministic information on the shipments is available over a substantial length of time. Of course, this depends on the related recognition and information systems being set up properly. This paper explores whether (and to what extent) the additional information on the queuing sequence of shipments (and the potential investment for retrieving this information) can contribute to increasing the sortation performance of an SC. For this purpose, we are the first to formulate the sorter scheduling problem, and providing appropriate scheduling algorithms is the major aim of this paper.

Prior to these operational planning tasks, an appropriate layout for the sortation system must be identified. For the core system, this requires a decision on the number of SC loops and their rotation direction, as well as the number of loading stations, trays, and outbound containers. Naturally, there is a basic trade-off between investment costs and the operational sortation performance among the layout alternatives. In order to quickly evaluate the latter, we can turn to an important body of literature dealing with performance analysis of closed-loop conveyor systems, including Bastani (1988), Bozer and Hsieh (2005), Johnson (1998), and Schmidt and Jackman (2000). Given mostly stochastic information (and some additional deterministic parameters) on the interarrival times of shipments at the loading stations and/or the service rates at the outbound stations (e.g., machines), these papers anticipate the (average) performance of closed-loop systems for different machining and sortation settings. The proposed methods mainly aim to quickly estimate system performance, e.g., the throughput, for different design parameters of SCs so that potential SC layouts can be compared. Reviews of these approaches in the literature are provided by Muth and White (1979) and Nazzal and El-Nashar (2007). However, these analytical methods can be applied only for relatively simple layout settings, such as those with a single loading station and a single loop. Properly evaluating the sortation performance of more complex layouts with multiple loading stations and bidirectional loops requires a solution for the operational decision tasks, and thus the scheduling procedures developed in this paper will be applicable for evaluating a specific sorter layout as well.

1.3 Research questions and paper structure

This paper is the first to consider the short-term decision problem of scheduling the loading process of shipments into an SC. Given the layout of an SC, which may differ in the number of loops, loading stations, and trays, this paper investigates the following fundamental scheduling decisions for a deterministic set of shipments to be sorted:

- Which direction? Whenever two parallel loops rotate in opposite directions, the direction in which a shipment is sent is a decision variable. A strategy of choosing the first empty tray in either direction or in the direction providing the shorter travel time to the respective outbound container may prove to be short-sighted. As outlined above,

this decision may affect the wait time of other shipments in queue for an empty tray.

- Which tray? Once a shipment is first in the queue of a loading station, it must be loaded onto an empty tray. Intuitively, loading it as soon as possible—i.e., on the first available tray passing by in the chosen direction—may seem to be a good choice. However, this may delay other shipments at successive stations, which cannot be put on the same tray. For example, a shipment approaching the main conveyor through the left loading station in Fig. 1, and having a destination container at the top of Fig. 1, blocks the tray it travels on while passing by the right loading station. With regard to overall optimal scheduling, it may be better to leave this tray empty, since shipments waiting in the right loading station are more important. Thus, making a decision regarding the tray a shipment occupies on a local level may not be the best strategy.
- Which loading station? Typically, an SC is part of a larger sortation system consisting of multiple SCs and/or storage areas interconnected by a system of switches and feeder lines. In such a setting, a given stream of shipments can be split up at a switch, and so the assignment of shipments to loading stations becomes an additional decision variable. For example, in Fig. 1, for each shipment, we may be able to decide through which loading station it approaches the loop of trays. This allows us to balance the number of shipments among loading stations. Note, however, that for various reasons, the loading station cannot be chosen in some systems (see Sect. 2.1). Therefore, we also consider problem versions with a predetermined loading station for each shipment.
- Which sequence? Lastly, the sequence of shipments reaching a loading station may also be alterable if, for instance, the retrieval sequence of goods stored in some automated storage and retrieval system or the unloading sequence from an inbound vehicle can be changed in order to accelerate the sortation process. Since, again, the ability to choose the sequence of shipments in any one loading station depends on the system at hand, we consider problem versions both with sequences still to be determined and with predetermined sequences.

Although, in the real world, these decisions are made with the help of very simple priority rules, the aim of this paper is to determine optimal solutions that minimize the makespan. In this way, we explore the question of whether the additional effort involved in updating a sortation system and associated information systems from simple real-time control rules to sophisticated (off-line) optimization procedures really pays. Our results show that (except for a single system configuration), simple control rules perform surprisingly well. Furthermore, the algorithms developed in this paper enable an evaluation of different layout alternatives with varying numbers of loading stations and loops. Thus, a second aim of this paper is to derive some basic design rules for SCs.

The remainder of the paper is structured as follows. First, we define our novel sorter scheduling problem and differentiate important subproblems (Sect. 2). We then present exact and heuristic procedure solutions for all of them (Sect. 3). In a comprehensive computational study (Sect. 4), the performance of these procedures is evaluated, and some basic layout rules for designing new sortation systems are derived. Finally, Sect. 5 concludes the paper.

2 Problem description

2.1 Single-level SCs

We consider a set $\mathcal{S}, \mathcal{S} = \{1, \ldots, S\}$, of shipments to be sorted by a single unidirectional SC consisting of a set $\mathcal{L}$, $\mathcal{L} = \{1, \ldots, L\}$, of loading stations, a set $\mathcal{T}$, $\mathcal{T} = \{1, \ldots, T\}$, of identical trays, and a set $\mathcal{C}, \mathcal{C} = \{1, \ldots, C\}$, of outbound containers. Shipments are successively loaded onto trays at their respective loading stations and travel along the loop until reaching their outbound containers. For convenience, we consider a set of periods $\mathcal{P} = \{1, \ldots, P\}$ discretizing the planning horizon. In each period, there is exactly one tray t located at each loading station l. The SC moves its trays continuously, such that in the next period, one of t's neighbors is located at station l. Note that it is predetermined which neighbor will be at l, since the rotation direction of an SC is given. We assume that the trays are moved such that they visit loading stations in increasing order of their numbers (and loading station 1 after loading station L). The positions of trays not located at a loading station in $p \in \mathcal{P}$ define a set of discrete tray positions. The set $\mathcal{C}$ of outbound containers is distributed around the loop, so that there is a container at each discrete tray position of the SC where no loading station is located. Thus, the number of containers C is given as $T - L$, and in any period, each tray is located either at a loading station or at a container. Note that, if there exist fewer containers in a real-world system, we can simply fill the set $\mathcal{C}$ with virtual containers receiving no shipments.

Shipments queue up in loading stations, and one item can be loaded in each period. Thus, in period p, the shipment s at the head of the queue of its respective loading station l can be loaded onto tray t located at l in p, if no other shipment is occupying tray t in period p. Then, s travels without interruption to its given destination container c. Thus, the period in which s reaches c is ultimately determined by the positions of l and c, the direction of the SC, and the period when s is loaded. Note that we can restrict ourselves to a set

$\mathcal{P}$ of periods, with $P = S \cdot T$ as long as postponing the processing of shipments cannot be beneficial.

Regarding the queues of shipments in loading stations, we consider four different settings:

1. The assignment of shipments to loading stations and the sequence of shipments in any loading station are predetermined. This situation arises whenever a random stream of shipments enters a loading station directly, without passing any presorting facilities. Many parcel distribution centers face such situations (see Fedtke and Boysen (2014)). Here, inbound trucks are assigned to free dock doors, typically according to a first-come/first-served policy. A worker then unloads parcels onto a conveyor belt, which directly feeds a dedicated loading station without passing any switches. This setting is also relevant whenever a complex sorting system is broken down into its elements, so that isolated SCs are successively considered while eliminating interdependencies.
2. The assignment of shipments to loading stations is given, but the sequence of shipments in the same loading station is still free. We encountered this setting in business practice at a parcel sorting facility of a major German postal service provider. Here, the main SC for sorting inbound and outbound mail arriving and leaving via trucks is directly interconnected with an automated storage and retrieval system (ASRS), which provides storage space for Internet retailers. Another example is early check-in or transfer baggage intermediately stored in an ASRS of an airport (Vanderlande 2014). In both examples, the retrieval sequence from the ASRS can be varied in order to appropriately sequence the shipments arriving at the loading stations of the sorter.
3. The assignment of shipments to loading stations is free, but the sequence of shipments in the same loading station is implied by an inbound sequence of all shipments. Hence, given that shipment s precedes another shipment s' in the inbound sequence, then s must still precede s' if both are assigned to the same loading station. This situation may arise if, for instance, an SC is fed via a single feeder conveyor, and a switch apportions the incoming stream for loading via multiple loading stations.
4. Here, both the assignment of shipments to loading stations and the sequence of shipments in the same loading station are free. This setting occurs naturally given the ones specified above.

The objective we pursue in this paper is minimizing the makespan, i.e., the latest arrival time of any shipment at its destination container. However, we will illustrate in Sect. 3 that some solution methods can be generalized to accommodate any regular objective function of shipment arrival times.

Next, we formally define the four problem settings introduced above.

1. We consider a given partition $\mathcal{S}_1, \ldots, \mathcal{S}_L$ of $\mathcal{S}$ and a given sequence σ_l of shipments for each loading station. Let $\sigma_l(k)$ denote the kth shipment in the sequence of the loading station l. Let $\Sigma \subset \mathcal{S} \times \mathcal{P}$ be a set of tuples (s, p) where the first entry specifies a shipment s and the second entry specifies the period p during which it is loaded onto the SC. We say that Σ is a schedule if
   - for each $s \in \mathcal{S}$, we have exactly one $(s, p) \in \Sigma$, i.e., each shipment is loaded exactly once,
   - for each pair of shipments s and s' with $\sigma_l(k) = s$ and $\sigma_l(k+1) = s'$, we have $(s, p) \in \Sigma$ and $(s', p') \in \Sigma$, with $p' > p$, i.e., queues in loading stations must be respected, and
   - for each pair of shipments s and s' with $\sigma_l(k) = s$ and $\sigma_{l'}(k') = s'$, with $l < l'$, we have $(s, p) \in \Sigma$ and $(s', p') \in \Sigma$, with $p' \neq p + \delta_{ll'}$ if s travels by l', and $p \neq p' + \delta_{l'l}$ if s' travels by l, i.e., shipments s and s' must not be on the same tray at the same time. Note that distance $\delta_{ll'}$ defines the number of periods required for a tray to travel from station l to l' in flow direction, which obviously depends on the given positions of the stations within the loop.

   Let $a_\Sigma(s)$ be the arrival time of shipment s according to Σ. Then, the single-level SC scheduling problem with fixed assignments of shipments to loading stations and fixed loading sequences—1-FA-FS for short—is to find a schedule

$$\sigma^* = \arg\min \{\max \{a_\Sigma(s) \mid s \in S\} \mid \Sigma \text{ is a schedule}\}.$$

2. A solution to the single-level SC scheduling problem with fixed assignments of shipments to loading stations and variable loading sequences—1-FA-VS for short—can be specified by a sequence σ_l of shipments assigned to l for each loading station l and a schedule Σ with respect to σ_l, $l \in \mathcal{L}$ according to 1-FA-FS. Again, the objective is to minimize the makespan.
3. A solution to the single-level SC scheduling problem with variable assignments of shipments to loading stations and fixed loading sequences—1-VA-FS for short—can be specified by a partition $\mathcal{S}_1, \ldots, \mathcal{S}_L$ of $\mathcal{S}$ representing the assignment of shipments to loading stations and a schedule Σ. Here, Σ is required to be a schedule with respect to $\bar{\sigma}_l$, $l \in \mathcal{L}$ according to 1-FA-FS, where $\bar{\sigma}_l$ is the sequence for $\mathcal{S}_l$ induced by the unique inbound sequence, i.e., if s precedes s' in the inbound sequence and s and s' are assigned to the same loading station, then s is loaded ear-

lier than s'. Again, the objective is to minimize the makespan.

4. Finally, a solution to the single-level SC scheduling problem with variable assignments of shipments to loading stations and variable loading sequences—1-VA-VS for short—can be specified by a partition $\mathcal{S}_1, \ldots, \mathcal{S}_L$ of $\mathcal{S}$ representing the assignment of shipments to loading stations, a sequence σ_l of shipments assigned to l for each loading station l, and a schedule Σ. Here, Σ is required to be a schedule with respect to σ_l, $l \in \mathcal{L}$ according to 1-FA-FS. Again, the objective is to minimize the makespan.

Analogously, the following section defines these problem settings for a bidirectional SC consisting of two loops.

2.2 Two-level SCs

We consider the set $\mathcal{S}$ of shipments to be sorted by an SC with two loops arranged one on top of the other, a set $\mathcal{L}$ of loading stations with given positions, and a set $\mathcal{C}$ of outbound containers occupying all remaining positions. We distinguish between the lower and upper levels rotating in opposite directions. Both levels are assumed to be identical except for their rotation direction, so that each level consists of the same number T of trays. Any pair of trays from different levels occupying the same slot (position) in the loop during any period p is connected with either the one loading station or the one outbound container located at the respective slot. Thus, in each period and each slot, exactly one shipment at a loading station can be launched in either direction (provided that trays are free), or up to two shipments can be tilted and reach the respective outbound container. As with the single-level problems, we must decide on the period during which a shipment is loaded onto a tray, as well as the direction that any shipment will travel. Once the start period and the direction of a shipment's travel are determined, the arrival time of this shipment is fixed as well, and we aim to minimize the makespan. As in the single-level case, we distinguish among the four settings discussed in Sect. 2.1, and refer to these problems as 2-FA-FS, 2-FA-VS, 2-VA-FS, and 2-VA-VS, respectively.

3 Algorithms

This section is dedicated to exact and heuristic algorithms solving the aforementioned one- and two-level SC scheduling problems. In Sect. 3.1, we develop exact algorithms for problems 2-FA-FS, 2-FA-VS, 2-VA-FS, and 2-VA-VS. The runtime complexity of three of these algorithms depends on the number of trays in the SC, and therefore the solution time may become quite large. Consequently, we provide strongly polynomial (exact) algorithms for special cases in Sect. 3.2 and introduce heuristic procedures in Sect. 3.3.

3.1 Exact algorithms for general settings

We first develop a dynamic programming (DP) approach for problem 2-FA-FS. Consider states $(p, (k_1, \ldots, k_L), (\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T))$, where

- p denotes the current period,
- k_l defines the number of shipments in loading station l already loaded onto the SC,
- $\bar{l}_t$ represents the last loading station the shipment currently occupying tray t in the upper level travels by, and equals 0 if the tray is free, and
- $\underline{l}_t$ denotes the last loading station the shipment currently occupying tray t in the lower level travels by, and equals 0 if the tray is free.

We assume that, in both levels, trays are numbered in increasing order of their first arrival time at loading station 1, so that during each period p, the position of each tray can easily be derived from its number. Beginning with initial state $(0, (0, \ldots, 0), (0, \ldots, 0), (0, \ldots, 0))$, there is a transition from state $(p, (k_1, \ldots, k_L), (\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T))$ to state $(p+1, (k'_1, \ldots, k'_L), (\bar{l}'_1, \ldots, \bar{l}'_T), (\underline{l}'_1, \ldots, \underline{l}'_T))$ if

- $\bar{l}'_t = \bar{l}_t$ ($\underline{l}'_t = \underline{l}_t$) if tray t in the upper (lower) level is not at a loading station in period p,
- $\bar{l}'_t = 0$ ($\underline{l}'_t = 0$) if $\bar{l}_t > 0$ ($\underline{l}_t > 0$) and tray t in the upper (lower) level is at loading station $\bar{l}_t$ ($\underline{l}_t$) in p,
- $\bar{l}'_t = l$ ($\underline{l}'_t = l$) if $\bar{l}_t = 0$ ($\underline{l}_t = 0$), t in the upper (lower) level is at loading station l' in p and l is the last loading station $\sigma_{l'}(k_{l'}+1)$ traveling by when loaded on the upper (lower) level,
- $\bar{l}'_t = \bar{l}_t$ or $\underline{l}'_{t'} = \underline{l}_{t'}$ if t in the upper level and t' in the lower level are the trays located at loading station l, $l \in \mathcal{L}$, in p,
- $k'_l = k_l + 1$ if $\bar{l}'_t > \bar{l}_t$ or $\underline{l}'_{t'} > \underline{l}_{t'}$ and t in the upper level and t' in the lower level are the trays located at loading station l, $l \in \mathcal{L}$, in p, and
- $k'_l = k_l$ if $\bar{l}'_t = \bar{l}_t$ and $\underline{l}'_{t'} = \underline{l}_{t'}$ and t in the upper level and t' in the lower level are the trays located at loading station l, $l \in \mathcal{L}$, in p.

A transition represents the choice of whether or not the next shipment is loaded on the lower or upper level of the SC during a period p. Note that we can mark a tray as free (second bullet point) once it travels by its last loading station before finally being tilted, because it cannot be loaded before

reaching the next loading station. Each transition (q, q') from state q to state q' implies a maximum arrival date $a^{\max}(q, q')$ of all those shipments whose loading is represented by this transition. We can then determine the minimum maximum arrival time $a^{\max}(q')$ among shipments loaded thus far for each state q' by applying a basic recursive formula

$$a^{\max}(q') = \min\left\{\max\left\{a^{\max}(q), a^{\max}(q, q')\right\} \mid (q, q') \text{ exists}\right\},$$

with $a^{\max}(0, (0, \ldots, 0), (0, \ldots, 0)) = 0$. We find the optimal schedule as $\arg\min\{a^{\max}(p, (k_1, \ldots, k_L), (\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T)) \mid k_l = |\mathcal{S}_l|, l \in \mathcal{L}\}$, i.e., the state having the shortest maximum arrival time with all shipments loaded.

To derive the runtime complexity of this DP in Lemma 1, we first introduce an upper bound on the number of periods P, which improves the aforementioned trivial bound $P \leq S \cdot T$.

Theorem 1 *There is an optimal solution where the last shipment in loading station l is loaded onto the SC no later than in period $2 \cdot |\mathcal{S}| - |\mathcal{S}_l|$.*

Proof See "Appendix 1". □

Lemma 1 *2-FA-FS can be solved by the DP defined above in $O(S^{L+1}L^{2T}3^L)$ time.*

Proof There are $O(S^{L+1}L^{2T})$ states due to Theorem 1. Each state is the origin of $O(3^L)$ transitions, since for each loading station, we can choose whether the next shipment is loaded on the upper or lower level or is not loaded at all during the current period. The lemma follows. □

It is easy to see that 1-FA-FS can be solved by a restricted version of the DP approach developed above in $O(S^{L+1}L^T2^L)$ time.

For the next two problem versions, namely 2-FA-VS and 2-VA-FS, we only sketch the approaches proposed, since they resemble the one developed above. For 2-FA-VS, we consider set $\mathcal{S}_{l,l'}$, $\mathcal{S}_{l,l'} \subseteq \mathcal{S}_l$, for each $l \in \mathcal{L}$ such that $s \in \mathcal{S}_{l,l'}$ if and only if its destination container is located between loading stations l' and $l' + 1$. Assume that on the upper level, shipments starting at l reach l' earlier than $l' + 1$. Let shipments in $\mathcal{S}_{l,l'}$ be numbered according to a non-decreasing distance of the positions of their destination containers to l'.

Lemma 2 *There is an optimal solution with an $s^* \in \mathcal{S}_{l,l'}$ such that s^* is either the first shipment starting from l on the upper level or the first shipment starting from l on the lower level, shipments $1, \ldots, s^* - 1$ are loaded on the upper level in decreasing order of their indices, and shipments $s^* + 1, \ldots, |\mathcal{S}_{l,l'}|$ are loaded on the lower level in increasing order of their indices.*

Proof See "Appendix 2". □

Lemma 2 allows us to apply the following technique. When building the loading sequence of shipments starting from l, we first choose the dividing element s^* for each subset $\mathcal{S}_{l,l'}$ and obtain two subsets. For both subsets, we can then assume the order in which they can be loaded and the level on which we can load them to be given. Therefore, we consider states

$$\begin{aligned}&(p, ((\overline{k}_{1,1}, \ldots, \overline{k}_{1,L}), \ldots, (\overline{k}_{L,1}, \ldots, \overline{k}_{L,L})),\\ &\quad((\underline{k}_{1,1}, \ldots, \underline{k}_{1,L}), \ldots, (\underline{k}_{L,1}, \ldots, \underline{k}_{L,L})),\\ &\quad(\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T))\end{aligned}$$

, where

- p denotes the current period,
- $\overline{k}_{l,l'}$ represents the last shipment from $\mathcal{S}_{l,l'}$ already loaded on the upper level of the SC,
- $\underline{k}_{l,l'}$ defines the last shipment from $\mathcal{S}_{l,l'}$ already loaded on the lower level of the SC,
- $\bar{l}_t$ denotes the last loading station the shipment currently occupying tray t in the upper level travels by, and equals 0 if the tray is currently free, and
- $\underline{l}_t$ defines the last loading station the shipment currently occupying tray t in the lower level travels by, and equals 0 if the tray is currently free.

Now we are ready to design a DP approach where we process period by period, deciding whether a shipment is loaded onto the SC at station l, $l \in \mathcal{L}$, and on which level it is loaded. Note that $\underline{k}_{l,l'}$ is decreased when a corresponding shipment is loaded on the lower level, and that $\overline{k}_{l,l'}$ is increased when a corresponding shipment is loaded on the upper level.

Lemma 3 *2-FA-VS can be solved in $O(S^{2L^2+1}L^{2T}3^L)$ time.*

Proof There are $O(S^{2L^2+1}L^{2T})$ states due to Theorem 1. Each state is the origin of $O(3^L)$ transitions, since for each loading station, there is a choice of whether the next shipment is loaded during the current period, is loaded on the upper level, or is loaded on the lower level. The lemma follows. □

1-FA-VS can be solved by a restricted version of the DP approach developed above in $O(S^{L^2+1}L^T2^L)$ time.

For 2-VA-FS we consider states $(p, k, (\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T))$ where

- p defines the current period,
- k denotes the number of shipments already loaded on the SC,
- $\bar{l}_t$ represents the last loading station the shipment currently occupying tray t in the upper level travels by, and equals 0 if the tray is currently free, and

- $\underline{l}_t$ denotes the last loading station the shipment currently occupying tray t in the lower level travels by, and equals 0 if the tray is currently free.

Note that the k shipments loaded in period p can be assumed to be the first k shipments in the inbound sequence. A transition from state $(p, k, (\bar{l}_1, \ldots, \bar{l}_T), (\underline{l}_1, \ldots, \underline{l}_T))$ to state $(p+1, k', (\bar{l}'_1, \ldots, \bar{l}'_T), (\underline{l}'_1, \ldots, \underline{l}'_T))$ represents loading the $k+1$th to k'th shipments of the inbound sequence at different loading stations during period p. This implies a decision about the assignment of these shipments to loading stations. Note that $k' - k \leq L$.

Lemma 4 *2-VA-FS can be solved in $O(S^2 L^{2T+1} L!)$ time.*

Proof There are $O(S^2 L^{2T})$ states due to Theorem 1. Each state may be the origin of $O(L \cdot L!)$ transitions, since there may be 1 to L shipments loaded in p, and there are up to $L!$ assignments of these shipments to loading stations. The lemma follows. □

1-VA-FS can be solved by a restricted version of the DP approach developed above in $O(S^2 L^{T+1} L!)$ time.

In order to solve 2-VA-VS, we first show that the set of loading stations for each shipment can be restricted.

Lemma 5 *There is an optimal solution where no shipment travels by a loading station.*

Proof See "Appendix 3".

Lemma 5 implies that for each shipment, we need to consider only two loading stations as candidates when choosing the shipment's actual loading station. Furthermore, it gives us the freedom to assume that every tray reaching a loading station is free. In addition, it is not difficult to see that for each loading station l and direction d, the set $\mathcal{S}_l^d$ of shipments loaded in l and going in direction d can be assumed to be loaded in non-increasing distance of their respective destination containers to l. Therefore, we consider sequences $\sigma_{l,l+1} = (s_1^l, \ldots, s_{n(l)}^l)$ of the jobs having destination containers between l and $l+1$ ordered in non-decreasing distance between destination containers and l. With reasoning analogous to that in Lemma 2, we can assume that there is an s^l in $\sigma_{l,l+1}$ such that all shipments preceding s^l are loaded at l, and all shipments following s^l are loaded at $l+1$.

Hence, we consider states

$$(p, ((\bar{k}_{1,2}, \underline{k}_{1,2}), (\bar{k}_{2,3}, \underline{k}_{2,3}) \ldots, (\bar{k}_{L,1}, \underline{k}_{L,1}))$$

, where

- p denotes the current period,
- $\bar{k}_{l,l+1}$ represents the last shipment from $\sigma_{l,l+1}$ loaded at l or the first shipment from $\sigma_{l,l+1}$ loaded at $l+1$ (in both cases its predecessor in $\sigma_{l,l+1}$ is the next shipment to be loaded at l), and
- $\underline{k}_{l,l+1}$ defines the last shipment from $\sigma_{l,l+1}$ loaded at $l+1$ or the first shipment from $\sigma_{l,l+1}$ loaded at l (in both cases its successor in $\sigma_{l,l+1}$ is the next shipment to be loaded at $l+1$).

Now we are ready to design a DP approach where we process period by period, deciding whether a shipment in $\sigma_{l,l+1}$ is loaded next in l, or a shipment in $\sigma_{l-1,l}$ is loaded next in l. Let $\sigma_{l,l+1}$ be chosen. If no shipment in $\sigma_{l,l+1}$ has yet been loaded, then there are $n(l)$ choices for the package to be loaded at l. However, if a shipment in $\sigma_{l,l+1}$ has already been loaded, then the shipment to be loaded at l has been determined. Again, from loading time and loading station we can derive the arrival time of each shipment.

Lemma 6 *2-VA-VS can be solved in $O(S^{3L+1})$ time.*

Proof There are $O(S^{2L+1})$ states due to Theorem 1. Each state may be the origin of $O(S^L)$ transitions, since for each loading station we can choose the next shipment to be loaded from a set of at most $O(S)$ shipments. The lemma follows. □

1-VA-VS can be solved in $O(S \log S)$ time, since we can still assume that no shipment passes a loading station, and the sequence of shipments in each loading station can be assumed to be known.

The results of this section are summarized by the following theorem.

Theorem 2 *Each problem introduced in Sect. 2 with L and T fixed can be solved in polynomial time.*

Note that each of the approaches developed in this section keeps track of the system period by period. Therefore, we can clearly identify the loading and arrival times of each shipment, allowing us to employ the same basic algorithms in order to solve problems having other objective functions as long as Theorem 1 holds, which requires a regular objective function.

3.2 Polynomial time exact algorithms for two loading stations

In this section, we consider a special case where SCs have only two equidistant loading stations. This means that the two loading stations are positioned such that they have maximum distance between each other, and each loaded shipment reaches the other loading station exactly after traveling a half circle. This setting is quite common in the postal distribution

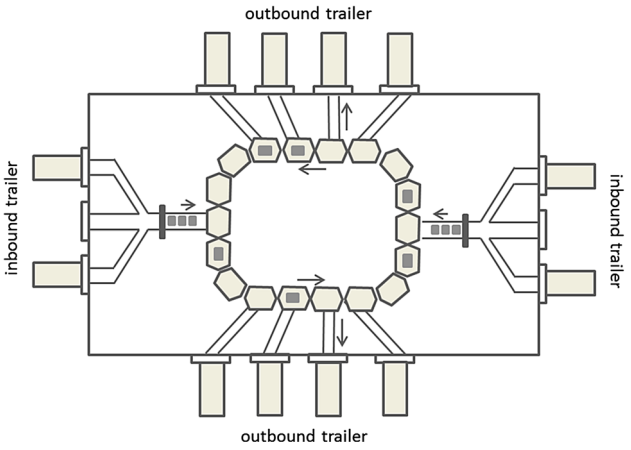

Fig. 2 Schematic layout of a standard German postal distribution center

centers we visited in Germany (see also Fedtke and Boysen (2014)). Typically, the longer sides of the rectangular terminal building are associated with outbound operations, and dock doors for processing inbound trucks are located on the narrower sides. All docks of an inbound side are connected to the same loading station, so that there are exactly two equidistant loading stations, as depicted in Fig. 2. For this special case, we derive solution procedures that are more efficient than those developed in Sect. 3.1. Note that a variable assignment (VA) of shipments to loading stations would require bypass conveyors that transport shipments from one side of the terminal to the loading station of the other side. We have not yet seen such a system configuration in a real-world terminal, but it is nonetheless technically possible.

Lemma 7 *There is an optimal solution to 2-FA-FS, 2-FA-VS, 2-VA-FS, and 2-VA-VS with two equidistantly positioned loading stations, where each shipment travels in the direction minimizing its makespan once its loading station is determined.*

Proof First, if each shipment travels the shorter route from its given loading station to its destination container, then no tray arrives in an occupied state at any loading station. This implies that the kth shipment in a given loading sequence can start in period k, which obviously cannot be undercut. Second, each shipment's travel time is minimized.

Since the arrival time of each package equals its start time plus its travel time, and we minimize both, we obviously minimize each shipment's arrival time and thus the objective value. □

Considering Lemma 7, it is easy to see that 2-FA-FS can be solved in $O(S)$, and 2-FA-VS is solvable in $O(S \log S)$, by sorting shipments at both loading stations in non-increasing order of travel distance.

For 2-VA-FS, we provide a DP and consider states (k_1, k_2), where k_1 and k_2 represent the number of shipments assigned to loading stations 1 and 2, respectively, among the first $k_1 + k_2$ first shipments in the unique inbound sequence. There is a transition from state (k_1, k_2) to states $(k_1 + 1, k_2)$ and (k_1, k_2+1), representing the assignment of the (k_1+k_2+1)th shipment of the inbound sequence to loading stations 1 and 2, respectively. Note that according to Lemma 7, we can assume that this shipment is launched in periods $k_1 + 1$ and $k_2 + 1$, respectively, and given that it travels the shorter of the two possible routes, its arrival time is determined as well.

Lemma 8 *2-VA-FS with two equidistantly positioned loading stations can be solved in $O(S^2)$ time.*

Proof There are $O(S^2)$ states, and each state may be the origin of $O(1)$ transitions. The lemma follows. □

For 2-VA-VS, let the shipments be numbered according to the non-decreasing distance of their destination container to loading station 1. Then, according to the reasoning used above, we can assume that shipment s is loaded before (after) s', $s' < s$, if both are assigned to loading station 1 (2) and are sent in the same direction. Furthermore, with interchange arguments similar to those used in the proof of Lemma 2, we can assume that there is a shipment s such that s is either the first shipment starting from 1 or the first shipment starting from 2; shipments $1, \ldots, s-1$ start from 1, and shipments $s+1, \ldots, S$ start from 2. Note that each tray arriving at a loading station is free, and therefore, an exchange of shipments between loading stations, retaining the respective loading period at each loading station, cannot lead to a conflict regardless of the direction they travel.

Thus, determining s and the loading station it is assigned to fully determines a solution, and the number of solutions to be evaluated is $2S$. Now, consider the solution where each shipment starts at loading station 1. In the following, we apply a binary search procedure in order to determine s such that the latest arrival time is minimized. In the first step, we choose $s = \lceil S/2 \rceil$ and assign it to loading station 1. We can determine the maximum arrival time in $O(S)$ time. Now assume, without loss of generality, that the latest arrival time among shipments starting at loading station 1 exceeds the latest arrival time among shipments starting at loading station 2. Obviously, increasing s cannot decrease the latest arrival time, since the shipment having the latest arrival time in the current solution will have the same arrival time for each higher value of s. Therefore, we can restrict our search to $[1, s-1]$ in order to find a better solution.

Lemma 9 *2-VA-VS with two equidistantly positioned loading stations can be solved in $O(S \log S)$ time.*

Proof Sorting shipments takes $O(S \log S)$ time. The binary search suggested requires $O(\log S)$ steps, and in each step, the current solution can be evaluated in $O(S)$ time. □

The results of this section are summarized by the following theorem.

Theorem 3 *Each problem introduced in Sect. 3.2 with two equidistantly positioned loading stations can be solved in polynomial time.*

3.3 Heuristics

As stated in Theorem 2, we have polynomial time algorithms if both the number of loading stations and the number of trays are fixed. However, even in small systems, the number of trays is easily double-digit. Among the algorithms developed in Sect. 3.1, only that for 2-VA-VS has a runtime complexity which is not exponential in the number of trays. This algorithm, however, has a runtime complexity which is exponential in the number of loading stations, e.g., $O(n^{13})$ for $L = 4$. Hence, we cannot expect any of these algorithms to be applicable if decisions are to be made in a short period of time.

3.3.1 Priority rules

In practice, priority rules are typically used. The following are the most common.

First free rule (FFR): The *first free* rule seeks to load one shipment per period and station, regardless of direction or distance, i.e., essentially randomly.
First free in shortest direction rule (FFSDR): The *first free in shortest direction* rule waits for the first empty tray in the direction that allows the shipment to reach its outbound container first. Hence, in any given period, some stations may load no shipments at all (despite this being possible).
Shortest distance rule (SDR): The *shortest distance* rule seeks to load one shipment per station during each period, preferably in the shortest direction. If the only choice for loading a shipment in a station is to use a direction other than the shortest (because the other tray is not empty), then this is chosen instead.
Earliest delivery rule (EDR): The *earliest delivery* rule seeks to load each shipment such that the minimum total delivery time (including travel and waiting time) is achieved. This implies that shipments are sometimes launched in the non-shortest direction if the shortest direction is blocked for a long period, or that no shipment is loaded at all in a given station at a given time because waiting is more efficient than launching a parcel in the "wrong"
direction.

3.3.2 Beam search

Finally, the proposed DP procedures can also be used as the basis for a beam search heuristic Lowerre (1976). Applying beam search requires the setting of a parameter θ (the beam width), which constrains the number of states that are further explored in each stage. In order to decide which states are pruned, states are prioritized according to their lower bound. The lower bound on the objective value of a state is calculated differently depending on whether the allocation of shipments to stations is predetermined:

Allocation to stations is given (FA): The lower bound is derived by applying FFSDR, i.e., the first free in shortest direction rule, on the remaining shipments, and assuming that each station can always load one shipment per period, thus disregarding the fact that no empty tray may be available in a given period. Since all shipments are always sent to their destination on the shortest possible route without delay (except the unavoidable delay induced by the sequence), the lower bound must always be at least as good as the ultimate objective value.
Allocation to stations is free (VA): The lower bound is derived by applying FFSDR on the remaining shipments and assuming that in each period, L shipments (or all remaining shipments if fewer than L are left) can be loaded, each at their "ideal" station (i.e., the one that leads to the shortest travel time), even if that would mean loading multiple shipments at the same station in the same period.

Note that this lower bound can also be incorporated into the dynamic programming procedures, thus making them bounded DPs. The solution obtained via the first free in shortest direction rule can serve as the initial upper bound.

4 Computational study

4.1 Instance generation

Since there are no established test data for the SC scheduling problem, we will now describe how we generated the instances used in our computational experiments. First, the characteristics of the sortation system must be defined. Systems in use in practice can vary greatly in size and scope, from small distribution centers, where the main conveyor is merely a few hundred meters long, to huge international mail hubs or airports, where the main conveyor can have a total length of several kilometers. To reflect this, we vary the parameters for instance generation according to Table 1.

A common tray on a tilt tray conveyor in typical parcel and mail hubs has a length of about 0.85m (Fedtke and

Table 1 Parameters for instance generation

Symbol	Description	Values
S	Number of shipments	10, 60
L	Number of loading stations	2, 3, 5, 10
T	Number of trays per main conveyor	30, 60
e	Number of main conveyors	1 (unidirectional), 2 (bidirectional)

Boysen 2014), although this number can be much larger in systems that handle bulkier cargo, such as those at airports. Two main conveyors (bidirectional), each with $T = 60$ trays, would thus translate to a total metric length of about 100–300 m, depending on tray length. Our sortation system has as many slots as it has trays per main conveyor—meaning, for example, that if there are $T = 60$ trays and $L = 10$ loading stations, there are $C = 60 - 10 = 50$ (potential) outgoing containers, among which the loading stations are equidistantly situated. Consequently, each shipment s, $s = 1, \ldots, S$, is assigned one target container $rnd(1, T)$, where $rnd(1, T)$ is a discrete uniformly distributed random integer from the interval given in brackets, except for values $i \cdot \lfloor T/L \rfloor + 1$, $\forall i = 0, \ldots, L - 1$, because those are loading stations and not outgoing containers. With regard to the measurement of time, according to our assumptions, the trays advance one slot per time unit, where one slot is as long as a tray. For example, if the main conveyor moves at a speed of 2 m/s, and a tray has a length of 1 m, then one time unit (or period) would correspond to half a second.

For those problem variants where the sequence needs to be predetermined (FA-FS and VA-FS), the shipments are given an arbitrary order σ. Then, for problem variants FA-FS and FA-VS, each shipment in σ is randomly assigned to a station such that up to L sequences σ_l are created, and if two shipments s and s' are both assigned to the same station l, and s comes before s' according to inbound sequence σ, then the same must also hold for σ_l. Note that in problem variant FA-VS, the sequence σ_l is facultative, and only the assignment is fixed, i.e., if a shipment s is in σ_l, then this shipment must be processed at station l but not in any particular order. Since each instance consists of S shipments with destinations, as well as an inbound sequence of these shipments (σ), plus a sequence for each station (σ_l) which is consistent with the overall inbound sequence, the same instance can be used for all eight problem variants (FA-FS, VA-FS, FA-VS, and VA-VS, each with uni- and bidirectional main conveyors), where the predetermined sequences are ignored if they are irrelevant for the specific problem.

For each combination of parameters S, L, and T from Table 1, we generated 20 instances as described above, leading to a total of 480 instances. In addition, in order to have a more challenging test bed, we generated 60 very large instances, where $S = 10{,}000$, $T = 500$, and $L \in \{2, 5, 10\}$, corresponding to an SC of more than 1 kilometer total length (in the bidirectional case). Finally, we also generated 40 small instances, where $S = 20$, $T = 6$, and $L \in \{2, 3\}$, which is just about the largest instance size that can still be solved to optimality by our exact procedures.

4.2 Numerical experiments

To test the performance of the proposed algorithms, and to address the research question outlined in Sect. 1.3, we implemented the algorithms in C# 5.0 and executed them on an $x64$ PC with an Intel Core i7-3770 3.4 GHz CPU and 8192 MB of RAM. In Sect. 4.2.1, we first focus on the performance of the approaches proposed in Sect. 3. In Sect. 4.2.2, we compare different sortation system configurations in order to elucidate what constitutes an efficient design.

4.2.1 Performance of algorithms and priority rules

Since neither memory nor runtime requirements are polynomially bounded for some algorithms, we instituted a time limit of 30 min and a memory limit of 4 GB of RAM. If an algorithm did not finish solving an instance within the allotted space or time, it was forcibly terminated. The beam width for the beam search heuristic was set to $\theta = 50$. Note that in a preliminary study, we tested the impact of beam width θ. As is to be expected, the solution quality increases with a larger beam width. However, the positive effect quickly diminishes; hence our choice, $\theta = 50$, seems to be an adequate compromise between runtime and solution quality.

Table 2 lists the optimal makespan, averaged over all solved instances, for the problem variants with two main conveyors, i.e., 2-FA-FS, 2-VA-FS, 2-FA-VS, and 2-VA-VS, in columns *FAFS*, *VAFS*, *FAVS*, and *VAVS*, respectively. Columns L, T, and S describe the characteristics of the instances as in Table 1. *# solved* denotes the number of instances (out of 20) that could be solved by the bounded dynamic programming schemes for *all* problem variants within the time and memory limits. Finally, the columns labeled *sec.* list the average CPU time, in seconds, that the bounded DP needed to reach the optimal solution; only instances that could be solved to optimality in all problem varieties are taken into account. Table 3 shows the same results for the single main conveyor variants (1-FA-FS, 1-VA-FS, 1-FA-VS, and 1-VA-VS).

Table 2 Average optimal results for the four problem variants with two main conveyors (bidirectional) obtained via bounded DP.

L	T	S	# solved	VAVS	VAFS	FAVS	FAFS	sec. (VAVS)	sec. (VAFS)	sec. (FAVS)	sec. (FAFS)
2	6	20	20	10.00	10.00	12.00	12.55	<0.1	<0.1	<0.1	<0.1
3	6	20	19	7.00	7.00	7.11	10.89	1.9	45.0	<0.1	0.2
2	30	10	20	7.75	9.75	13.75	16.45	<0.1	<0.1	<0.1	<0.1
2	30	60	20	30.00	33.40	33.80	43.85	<0.1	<0.1	<0.1	<0.1
2	60	10	20	13.75	16.40	27.25	29.45	<0.1	<0.1	<0.1	<0.1
2	60	60	20	30.30	39.15	34.15	56.05	<0.1	<0.1	<0.1	<0.1
3	30	10	6	5.83	7.83	14.00	15.83	<0.1	62.6	<0.1	<0.1
3	60	10	7	9.57	11.14	28.00	29.57	<0.1	18.0	<0.1	<0.1
5	60	10	1	5.00	6.00	28.00	28.00	<0.1	<0.1	<0.1	<0.1
10	60	10	2	3.00	3.00	28.00	29.00	<0.1	<0.1	<0.1	<0.1

Table 3 Average optimal results for the four problem variants with one main conveyor (unidirectional) obtained via bounded DP

L	T	S	# solved	VAVS	VAFS	FAVS	FAFS	sec. (VAVS)	sec. (VAFS)	sec. (FAVS)	sec. (FAFS)
2	6	20	20	12.25	13.65	16.25	18.30	<0.1	<0.1	<0.1	<0.1
3	6	20	14	9.36	10.21	12.15	16.64	<0.1	0.1	37.5	<0.1
2	30	10	20	13.25	16.25	26.80	29.35	<0.1	0.2	<0.1	<0.1
2	60	10	20	26.65	30.30	54.05	57.90	<0.1	0.4	<0.1	<0.1
3	30	10	20	9.20	11.80	26.55	27.95	<0.1	53.5	<0.1	<0.1
3	60	10	13	18.38	22.31	56.77	58.62	<0.1	138.0	<0.1	<0.1

The tables show that the time and, in particular, the memory consumption of the DP procedures can be quite substantial, since many of the larger instances ($L > 2$) could not be solved to optimality for all problem variants. The very large instances ($S = 10,000$) are completely out of the DP's reach; not even the derived BS heuristic can solve them, given the exponentially growing memory requirements. It is also striking that the instances were often more difficult to solve in the single main conveyor cases, even though the solution space was greatly reduced. This can be explained by the fact that the lower bounds work less well in the single main conveyor case, since they abstract from blocked trays; trays are much more likely to be occupied for a longer time if there is only one instead of two main conveyors. Conversely, this allows us to draw the conclusion that at least in the bidirectional problem versions, the proposed lower bounds seem to be quite tight.

Another interesting issue revealed by the tables is the restrictiveness of the individual problem variants. It is not at all surprising that the least restrictive VA-VS systems perform best and the most restrictive FA-FS systems perform worst in terms of makespan; however, that VA-FS systems should sort shipments much more quickly than FA-VS systems, almost without exception, is not immediately obvious. Apparently, put to the choice, being restricted to a predetermined sequence (VA-FS) is greatly preferable to having to adhere to a predetermined station assignment (FA-VS). Indeed, the average makespan gap over all optimally solved instances between these two problem variants is quite substantial, at about 32.9 %, whereas the difference between a completely unrestricted system (VA-VS) and one where the sequence is given (VA-FS) is only about 17.2 %, and that between a system where the allocation of shipments to stations is fixed (FA-VS) and a fully restricted system (FA-FS) is only about 11.6 %. The poor performance of the FA problem variants can be explained by the potentially long distances the shipments may have to travel, causing much additional blockage at the intervening stations which the shipments must pass, which is especially severe in systems with a unidirectional main conveyor. This has some managerial implications: when setting up a new unidirectional SC system, emphasis should be placed on avoiding fixed station assignments. Additional investment in equipment and processes allowing greater flexibility in rearranging the sequence of incoming shipments, however, may not be worthwhile, as this makes comparatively little difference regardless of whether the station allocation is predetermined. Note, however, that the difference between VA-FS and FA-VS is less pronounced if the system is bidirectional and well utilized (i.e., $S > T$), because on the one hand, bidirectional main conveyors greatly reduce the average travel distance for FA systems, while on the other hand, high utilization can force VA-FS systems to

load shipments at remote stations just to advance in the sequence.

Figure 3 further explores the performance of the exact algorithms. Since the runtime is a far lesser problem than the memory requirements, comparing CPU times is not necessarily meaningful. Instead, the graphs show the number of instances (out of 20) per parameter constellation where the dynamic programming procedures exceeded their time and space limits. As expected, all the algorithms have trouble with the larger instances; however, the parameters affect the procedures differently. All algorithms scale poorly with the number of shipments, but for the FA-type problems (fixed assignment), the performance is generally better with regard to the number of loading stations. Recall that none of the DP-based procedures—including beam search—could solve the very large instances ($S = 10{,}000$).

For these very large instances, and indeed for most practical purposes, priority rules are applied. We introduced four of the most popular in Sect. 3.3: first free (FFR), first free in shortest direction (FFSDR), shortest distance (SDR), and earliest delivery (EDR). Their optimality gaps (and, for reference, that of beam search) are listed in Table 4, averaged over all instances that could be solved to optimality (where $T \geq 10$), and separated by problem setting. Unsurprisingly, the very simple first free rule does not perform at all well in any but the very simplest settings (fixed assignment, only one main conveyor). The other three rules, however, deliver passable results in many cases, especially FFSDR and EDR. SDR, which unlike FFSDR does not always send shipments in the shortest direction, performs notably worse than the other two rules, indicating that it is often better to wait for the tray in the shortest direction to become available than to load a shipment regardless of the difficulties. There is one—not immediately obvious—exception to this rule, however. When, for a bidirectional system, the sequence is given but the assignment is not (2-VA-FS), holding back a shipment to wait for a "better" tray means that all subsequent shipments in the sequence will have to wait as well. In the more difficult instances where trays are a scarce resource ($S > T$), it can be advantageous, therefore, to load trays even if the travel distance is not ideal. Consequently, the optimality gap for SDR in those cases ($S > T$) is merely 16.7 % on average, as opposed to 34.6 % for EDR. It stands to reason that this gap will shrink even further when S increases. Nonetheless, considering the comparatively large optimality gaps of all priority rules for the 2-VA-VS case, it certainly seems that these systems are quite sensitive to the quality of the schedule and may require more sophisticated algorithms, like BS. Finally, the table reveals that the beam search heuristic is quite powerful, creating solutions very close to optimal. The CPU time is also acceptable; only in the medium-sized instances ($S = 60$) does the runtime sometimes exceed 1 min.

Table 4 Average optimality gaps for the small and medium-sized instances. Only instances that could be solved to optimality are taken into account

e	FFR (%)	SDR (%)	FFSDR (%)	EDR (%)	BS (%)
(a) VA-VS					
1	102.32	54.68	0.00	11.74	0.00
2	309.14	45.17	4.83	0.61	4.82
(b) VA-FS					
1	59.99	43.03	0.47	0.47	0.69
2	103.05	44.14	11.88	11.62	0.08
(c) FA-VS					
1	0.00	0.00	0.00	0.00	0.00
2	53.12	49.02	0.82	0.82	0.00
(d) FA-FS					
1	0.00	0.00	0.00	0.00	0.00
2	75.70	4.28	0.00	0.00	0.00

As was mentioned previously, practitioners often use simple priority rules to sort their incoming shipments. Our study shows that in many cases, this is not a bad policy, as the optimality gaps are not overly large. There are, however, differences between the respective rules, and not all of them work equally well for all problem settings, as visualized by the bar graphs in Fig. 4. The bars show the makespan attained by the solutions found by the respective priority rules. They clearly indicate that the comparison between FFSDR and EDR is too close to call, and both are far superior to a random assignment of shipments to stations (FFR), and also to SDR in most cases. This may seem somewhat counterintuitive, because it may appear desirable to load as many shipments as possible in each period (as SDR does), regardless of whether they can be sent in the shortest direction. However, due to the many additional blockages along the way, this policy is rarely optimal. Interestingly, the sole exception to this is the 2-VA-FS case. Since an unloaded shipment blocks all following shipments in the sequence, SDR is in fact superior to all other rules for this setting. The take-home message here is that the chosen priority rule should be matched to the specific sortation system in place, because the differences in makespan can be quite substantial, even among the "good" priority rules (SDR, FFSDR, EDR), sometimes exceeding 30 %. If the right rule is chosen, however, the system often operates close to its theoretical optimum—given ten stations, 10,000 shipments cannot possibly be processed in fewer than 1000 periods.

With regard to the questions on how to derive schedules for the different SC settings defined in Sect. 1.3, we can draw the following conclusions. The exact methods developed in Sect. 3 have their limits when it comes to solving larger instances. The priority rules, naturally, yield solutions in much shorter runtime. What came as a surprise to us is

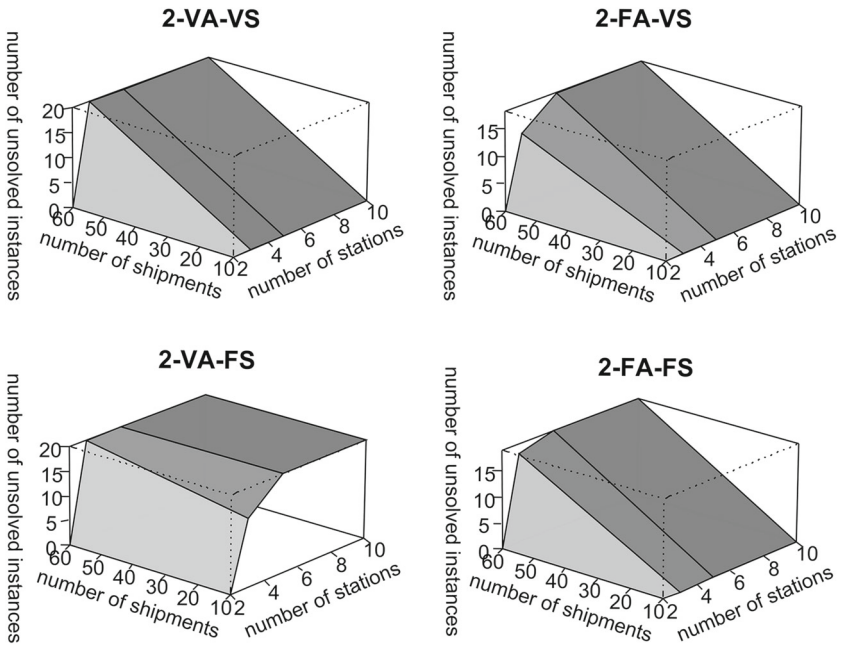

Fig. 3 Number of instances (out of 20) that could not be solved within the time and space limit by bounded DP ($T = 30, e = 2$)

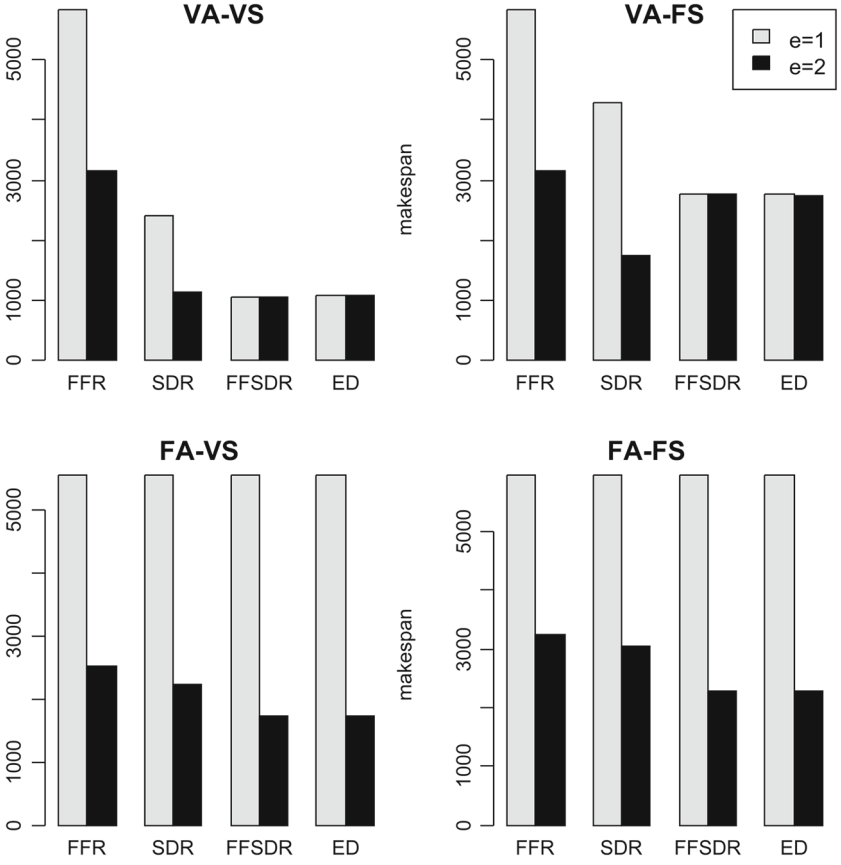

Fig. 4 Performance of the priority rules for the very large instances ($L = 10, T = 500, S = 10{,}000, e = 2$)

Table 5 Comparison of uni- and bidirectional conveyor systems in terms of relative makespan gap; results are obtained either via BS (small instances) or via the respective best priority rule (large instances)

L	T	S	VAVS (%)	VAFS (%)	FAVS (%)	FAFS (%)
2	30	10	72.34	69.57	96.10	80.16
2	30	60	8.67	49.51	45.74	55.41
3	30	10	55.54	35.46	87.01	80.12
3	30	60	−2.51	13.67	91.80	76.01
5	30	10	29.92	34.25	92.48	88.44
5	30	60	−4.99	9.23	109.32	84.52
10	30	10	21.92	–	97.31	95.63
10	30	60	0.41	–	–	127.53
2	60	10	96.35	87.57	99.62	98.78
2	60	60	16.19	61.04	73.47	45.63
3	60	10	94.37	76.97	106.87	101.55
3	60	60	7.68	18.43	84.52	62.10
5	60	10	76.10	51.74	108.40	102.55
5	60	60	9.00	18.32	96.40	77.86
10	60	10	63.75	–	95.29	92.87
10	60	60	−3.04	–	100.22	100.40
2	500	10,000	−0.02	44.09	49.28	50.51
5	500	10,000	0.15	74.84	152.85	109.76
10	500	10,000	−0.21	84.32	221.79	161.73

Table 6 Average reduction in makespan if $L = 5$ instead of $L = 2$ loading stations are used; results are obtained either via BS (small instances) or via the respective best priority rule (large instances)

T	S	2-VA-VS (%)	2-VA-FS (%)	2-FA-VS (%)	2-FA-FS (%)
30	10	84.92	71.67	−3.48	7.35
30	60	78.40	30.85	60.22	41.15
60	10	132.00	99.03	2.22	5.65
60	60	91.29	39.19	13.20	33.50
500	10,000	146.09	59.89	110.26	70.75

that, using the best of our priority rules, we can achieve near-optimal solutions. Our study provides a sense of which priority rule is particularly suited for a given sortation system setting.

4.2.2 Evaluation of sortation system configurations

Tables 5 and 6 compare different sortation system configurations with respect to the makespan achieved by applying either BS or the best priority rule. Table 5 examines the speedup attainable by applying a bidirectional rather than a single conveyor system for all four problem variants (columns *VAVS*, *FAVS*, *VAFS*, and *FAFS* in the table). The values are calculated as $(Z^1 - Z^2)/Z^2$, where Z^1 is the objective value (makespan) in the single conveyor case, and Z^2 is the makespan in the two-conveyor case, as reported by either the beam search heuristic (for the small/medium-sized instances) or the best priority rules (for the very large instances), averaged over all instances per parameter constellation.

The relative benefit of installing a second conveyor can be quite significant, especially for those settings where the assignment/sequence is in any way predetermined, as it shortens the average travel times substantially. For the completely unrestricted systems (VA-VS), however, the picture is somewhat different. Since in these systems each shipment can always be routed to its "ideal" station anyway, a second conveyor does not necessarily make a great difference. Only in the small instances with plenty of trays ($T > S$) is any kind of speedup noticeable. In the more realistic case of full utilization, the gap is negligible. This is certainly interesting, seeing that setting up a flexible VA-VS system is fairly costly to begin with. Once such a setup exists, however, no further investment in a bidirectional conveyor is necessary. In some cases, therefore, it may be advantageous to invest in flexibility with regard to sequence and assignment rather than a bidirectional conveyor.

Similarly, Table 6 shows the average relative reduction in makespan if $L = 5$ instead of only $L = 2$ loading stations are installed. The data clearly indicate that transshipment

Table 7 Suitability of priority rules

	e	VS	FS
VA	1	FFSDR	FFSDR/EDR
	2	EDR	SDR
FA	1	FFR/SDR/FFSDR/EDR	FFR/SDR/FFSDR/EDR
	2	FFSDR/EDR	FFSDR/EDR

is significantly accelerated if additional loading stations are used—as long as the added flexibility of these additional stations is actually put to good use. All sortation systems benefit to some degree from the additional capacity, but the effect is greatest for the VA-VS systems, which can flexibly assign the shipments to the new stations as required. For these systems, there is almost a linear relationship between the number of stations and the makespan, i.e., doubling the station count about halves the makespan. For the more restricted cases, especially those with fixed station assignment (FA), the effect is far weaker. Hence, we can conclude that an investment to increase the number of loading stations clearly pays off in VA systems, or alternatively in FA systems with a comparatively high degree of utilization, since the table suggests that, unsurprisingly, the greater the number of shipments, the greater the benefit of having extra stations.

To conclude this section, it can be said that the effect of additional resources, i.e., additional conveyor loops and loading stations, is heavily dependent on the system configuration of the SC, and thus a careful system design seems advisable in real-world applications.

5 Conclusions and Outlook

This paper considers a deterministic SC scheduling problem, where shipments queuing in loading stations of a closed-loop sorting system are to be successively loaded onto tilt trays such that the makespan of sorting items into outbound containers is minimized. Different subproblems are derived by either fixing the assignment of shipments to loading stations (fixed assignment, FA) or making the assignment decision part of the scheduling problem (variable assignment, VA). Additionally, the sequence of shipments in the loading stations may be either fixed (fixed sequence, FS) or variable (variable sequence, VS). Finally, we differentiate systems having only a single loop of trays ($e = 1$) and those consisting of two parallel loops rotating in opposite directions ($e = 2$). For all eight problem settings, exact and heuristic solution procedures are presented. In a comprehensive computational study, the following key findings are derived:

- Simple rules of thumb often applied in the real world, e.g., the earliest delivery rule, are sufficient in most problem settings, and produce only a minor optimality gap, less than 1 %. However, not just any rule fits any sortation system, and the assignment of (suitable) rules and settings summarized in Table 7 should be considered. Only in bidirectional sorting systems with a variable station assignment (VA) and fixed loading sequence (FS) is there the danger of a considerable optimality gap, about 11 %, where a sophisticated optimization procedure might be the better choice.
- In designing a sorting system, investment is better spent on switches in order to enable a variable assignment of shipments to loading stations (VA), instead of integrating presorting facilities or varying the retrieval sequence from a connected automated storage and retrieval system (VS).
- The increase in sorting performance when extending a single loop to a bidirectional system is considerable throughout all problem settings, with the exception of VA-VS, because their flexibility allows the assignment of shipments to the closest stations. Hence, bidirectional systems are most useful in systems with fixed station assignment (FA).
- Additional loading stations lead to a notable increase in sortation performance whenever the assignment of shipments to loading stations is variable (VA). With a predetermined assignment (FA), performance gains are less distinct, because additional stations lead to additional blockage whenever a poor assignment induces longer travel distances.

Future research should extend our isolated view on single SCs to a holistic view on more complex systems with multiple interconnected loops. Comparing the sorting performance of sophisticated scheduling procedures to simple real-world rules of thumb for more complex systems would be a valuable contribution for the practitioner when choosing a suitable scheduling policy. Moreover, additional restrictions, such as shipment arrival times, and other objective functions such as minimizing the lateness of multi-item orders bound to due dates (see Johnson 1998; Meller 1997) should be integrated into our novel SC scheduling problem.

Acknowledgements This research has been supported by the German Science Foundation (DFG) through the grant "Planning and operating sortation conveyor systems" BO 3148/5-1 and BR 3873/6-1.

Appendix 1: Proof of Theorem 1

Proof For simplicity, we say that the predecessor of the first shipment in l starts in period 0. For each shipment s, $s \in \mathcal{S}_l$, starting its travel in period p, we consider

- the immediate predecessor of s in l starting in period p',

- the set D_s of shipments in loading station l', $l' \neq l$, such that s travels by l', s, and shipments in D_s travel in the same direction, and for each $s' \in D_s$ there is a period p'', $p' < p'' < p$, such that s and s' would be on the same tray if s would start in p'', and
- the set D'_s of shipments in loading station l', $l' \neq l$, traveling by l in a period p'', $p' < p'' < p$.

Note that s must be loaded later than its predecessor and that D_s and D'_s prevent us from simply loading s in an earlier period but still after its predecessor. We refer to D_s and D'_s as delaying loaded shipments and delaying passing shipments, respectively, of s. Note that for shipments s and s', $s, s' \in \mathcal{S}_l$, $s \neq s'$, we have $D_s \cap D_{s'} = \emptyset$ and $D'_s \cap D'_{s'} = \emptyset$. Intuitively speaking, this means that each shipment in loading station l' can be a delaying shipment for any shipment in l at most twice: once as a delaying loaded shipment and once as a delaying passing shipment.

Now, assume that $\sigma_l(|\mathcal{S}_l|)$ is loaded onto the SC in period p, $p > 2 \cdot |\mathcal{S}| - |\mathcal{S}_l|$. There must then be a shipment s in l starting in p such that its predecessor starts in p' with $p - p' > |D_s| + |D'_s|$, since

$$\sum_{s \in \mathcal{S}_l} \left(|D_s| + |D'_s| \right) \leq 2 \cdot (|\mathcal{S}| - |\mathcal{S}_l|) .$$

Shipment s, then, can be loaded earlier in a period p'', $p' < p'' < p$, for which no delaying shipment of s exists. Clearly, the objective value is not increased by this modification. Note that each shipment can be scheduled earlier only a finite number of times. Therefore, finally, the last shipment will be loaded earlier than in period $2 \cdot |\mathcal{S}| - |\mathcal{S}_l| + 1$. □

Appendix 2: Proof of Lemma 2

Proof First, it is not difficult to see by an interchange argument that shipments starting from l on the upper level are loaded in decreasing order of their indices (and therefore in non-increasing order of their travel times), and shipments starting from l on the lower level are loaded in increasing order of their indices (and therefore in non-increasing order of their travel times).

Now, assume that for the first shipment s loaded on the upper level in period p, and the first shipment s' loaded on the lower level in p', we have $s > s'$. Exchanging s and s', i.e., loading s on the lower level in p' and loading s' on the upper level in p, cannot increase the objective value. Afterwards, we can sort shipments in both sequences according to their indices. This can be repeated until for the first shipment s loaded on the upper level in period p and the first shipment s' loaded on the lower level in p', we have $s < s'$. We can now choose s^* from one of the two sequences; if one is empty, we choose s^* from the other one. □

Appendix 3: Proof of Lemma 5

Proof Consider an arbitrary solution where χ, $\chi > 0$, is the number of times a shipment travels by a loading station. Consider the last period where a shipment s travels by a loading station l. Assume w. l. o. g. that s is traveling toward $l+1$ and that s starts in loading station l', $l' < l$. Clearly, s is occupying the tray located at l in p, and no shipment from l can be loaded on this tray. We distinguish between two cases in the following.

First, if no shipment from l is loaded in p going in the other direction, we can remove s from the sequence at loading station l', insert it into the sequence at loading station l such that it is ready to be loaded in period p, load it onto the same tray it occupies in the original solution (thus load it at l in period p), and obtain a solution where no shipment reaches its destination container later than in the original solution, and χ has been decreased.

Second, if a shipment s' from l is loaded in p going in the other direction, then let $p' = p - (l - l')T/L$ be the period where s is loaded at l'. We then remove s and s' from the sequence at loading stations l' and l, respectively, and insert s and s' into the sequence at loading stations l and l', such that it is ready to be loaded in p and p'. Note that there is no conflict with any other shipments loaded at l or l', since now s has the position formerly held by s', and vice versa. We load s onto the same tray it occupies in the original solution (thus load it at l in period p), and load s' this way as well (thus load it at l' in period p'). Note that the destination container of s' must be located between $l-1$ and l, since otherwise, s would not be the last shipment traveling by a loading station. Hence, sending s' from l' in the same direction as s is sent in the original solution is feasible, since this tray is not occupied when it reaches l'. Furthermore, s' reaches its destination container earlier than does s, and s reaches its destination container during the same period as in the original solution. Therefore, we obtain a feasible solution without increasing the objective value. Finally, s travels by no loading station in the new solution, and s' travels by one fewer loading station than does s in the original solution. Therefore, we decrease the number χ.

Repeating the step described above, we can obtain a solution with $\chi = 0$ which has an objective value not exceeding that of the original solution. □

References

Abdelghany, A., Abdelghany, K., & Narasimhan, R. (2006). Scheduling baggage-handling facilities in congested airports. *Journal of Air Transport Management*, *12*, 76–81.

Asco, A., Atkin, J., & Burke, E. (2014). An analysis of constructive algorithms for the airport baggage sorting station assignment problem. *Journal of Scheduling*, *17*(6), 601–619.

Bastani, A. S. (1988). Analytical solution of closed-loop conveyor systems with discrete and deterministic material flow. *European Journal of Operational Research*, *35*(2), 187–192.

Boysen, N., & Fliedner, M. (2010). Cross dock scheduling: Classification, literature review and research agenda. *Omega*, *38*, 413–422.

Bozer, Y. A., & Carlo, H. J. (2008). Optimizing inbound and outbound door assignments in less-than-truckload crossdocks. *IIE Transactions*, *40*(11), 1007–1018.

Bozer, Y. A., & Hsieh, Y.-J. (2005). Throughput performance analysis and machine layout for discrete-space closed-loop conveyors. *IIE Transactions*, *37*(1), 77–89.

Clausen, U., Diekmann, D., Baudach, J., Kaffka, J., & Pöting, M. (2015). Improving parcel transshipment operations – impact of different objective functions in a combined simulation and optimization approach. In *2015 Winter Simulation Conference (WSC)*, pages 1924–1935 .

DHL. (2008). Daten & Fakten. http://www.dp-dhl.com/content/dam/logistik_populaer/leipzig_hub/daten_fakten_hub_leipzig_de.pdf.

Fedtke, S., & Boysen, N. Layout planning of sortation conveyors in parcel distribution centers. *Transportation Science*, (to appear).

Gue, K. (1999). The effect of trailer scheduling on the layout of freight terminals. *Transportation Science*, *33*, 419–428.

Haneyah, S., Schutten, J., & Fikse, K. (2014). Throughput maximization of parcel sorter systems by scheduling inbound containers. In U. Clausen, M. ten Hompel, & F. Meier (Eds.), *Efficiency and innovation in logistics* (pp. 147–159). Berlin, Heidelberg: Springer International Publishing.

Johnson, M. E. (1998). The impact of sorting strategies on automated sortation system performance. *IIE Transactions*, *30*(1), 67–77.

Johnson, M. E., & Lofgren, T. (1994). Model decomposition speeds distribution center design. *Interfaces*, *24*(5), 95–106.

Lowerre, B. (1976). *The Harpy speech recognition system*. Ph.D. thesis, Carnegie Mellon University.

Meller, R. D. (1997). Optimal order-to-lane assignments in an order accumulation/sortation system. *IIE Transactions*, *29*(4), 293–301.

Muth, E. J., & White, J. A. (1979). Conveyor theory: A survey. AIIE. *Transactions*, *11*(4), 270–277.

Nazzal, D., & El-Nashar, A. (2007). Survey of research in modeling conveyor-based automated material handling systems in wafer fabs. In *Simulation Conference, 2007 Winter* (pp. 1781–1788).

Schmidt, L. C., & Jackman, J. (2000). Modeling recirculating conveyors with blocking. *European Journal of Operational Research*, *124*, 422–436.

Tsui, L. Y., & Chang, C.-H. (1990). A microcomputer based decision support tool for assigning dock doors in freight yards. *Computers & Industrial Engineering*, *19*, 309–312.

Tsui, L. Y., & Chang, C.-H. (1992). Optimal solution to a dock door assignment problem. *Computers & Industrial Engineering*, *23*, 283–286.

U.S. Department of Transportation. (2009). Air travel consumer report. http://airconsumer.ost.dot.gov/reports/2009/June/200906ATCR.PDF.

Vanderlande. (2014). Bagstore, the flexible, reliable and efficient storage solution. http://www.vanderlande.com/en/Baggage-Handling/Products-and-Solutions/Early-Bag-Storage/BAGSTORE.htm.

Werners, B., & Wülfing, T. (2010). Robust optimization of internal transports at a parcel sorting center operated by deutsche post world net. *European Journal of Operational Research*, *201*, 419–426.